Controlled Behavioral Language (CBL) – DRAFT

A Structured Specification Language for
Model-Based Product Design

Raktim Bhattacharya
Aerospace Engineering, Texas A&M University.

June 1, 2026

Abstract

This document provides a self-contained overview of the Controlled Behavioral Language (CBL): its motivation, syntax, formal semantics, design principles, trust architecture, worked examples, a prototype toolchain, and planned future directions. CBL is a structured specification language that addresses the gap between natural-language requirements and model-based product design (MBPD) toolchains. It enables engineers to write behavioral specifications that are readable, formally checkable, and compilable to industrial modeling targets (Stateflow, SCADE). Large language models assist in requirements elicitation and CBL inference; they do not generate code. Code generation is delegated to qualified MBPD tools, preserving the trust chain on which safety-critical certification depends.

Contents

1 Why CBL Was Developed

1.1 The Model-Based Product Design Pipeline

Model-Based Product Design (MBPD) for safety-critical embedded systems follows a well-established pipeline:

$$\text{Requirements} \xrightarrow{\text{manual}} \text{Model (Simulink/Stateflow/SCADE)} \xrightarrow{\text{qualified generator}} \text{Code} \xrightarrow{\text{V\&V}} \text{Certified System} \tag{1}$$

The later stages of this pipeline are mature. Simulink Coder and SCADE KCG are qualified code generators under DO-178C [16] and ISO 26262 [8]. MIL, SIL, and HIL testing provide systematic verification at successive integration stages. Formal methods tools (Simulink Design Verifier, SCADE Prover) can verify model-level properties. The trust chain from model to certified code is industrially deployed.

1.2 The Requirements-to-Model Gap

The first step, translating requirements into models, is where this trust chain breaks. Requirements arrive as natural-language documents: they are ambiguous in phrasing, incomplete in coverage, and often internally inconsistent. Engineers interpret these documents and manually construct models. This translation is unverified in any formal sense.

Studies consistently report that 40 to 60 percent of safety-critical software defects originate in requirements and early design, not in coding or integration [11]. These defects are injected during the manual requirements-to-model step that the rest of the MBPD pipeline cannot catch, because the pipeline verifies fidelity to the *model*, not fidelity to the *intent*.

The requirements-to-model step introduces three categories of defect:

1. **Incompleteness:** the model omits behaviors that the requirements describe or imply. Missing transitions, unhandled input combinations, and undefined corner cases.
2. **Inconsistency:** the model contains behaviors that contradict the requirements, arising from ambiguous natural language permitting multiple valid interpretations.
3. **Unintended behaviors:** the model includes behaviors that no requirement addresses, arising from modeling choices (default transitions, initial conditions, unguarded paths) that introduce behavior by construction rather than by specification.

1.3 Inadequacy of Existing Approaches

Filling the requirements-to-model gap requires four properties simultaneously:

(a) readability for domain engineers who are not formal-methods specialists,
(b) a mode-transition structure with decidable well-posedness conditions,
(c) declarative timing (persistence, deadlines) without manual counter management, and
(d) compilation to industrial MBPD targets (Stateflow, SCADE) with a clear tool qualification argument.

No existing approach provides all four. **EARS** [12] offers sentence templates but defines no formal semantics, mode structure, or consistency checking. **SCR** [6] provides mode-transition tables with decidable well-posedness but predates modern MBPD toolchains and is far from natural language. **FRET** [5] maps structured English to temporal logic for individual properties but lacks mode structure and compilation to executable models. **Contract-based design** [3] provides the theoretical framework for assume-guarantee reasoning but defines no concrete syntax or tooling. **Allium** [9] shares the premise that a structured behavioral artifact should precede implementation, but has no compiler, no formal semantics, and no verification layer: the LLM translates the specification directly to implementation code. CBL also uses an LLM, but only to produce the `.cbl` file; a deterministic well-posedness checker and a qualified code generator stand between the LLM's output and any executable artifact.

The broader movement toward LLM-assisted formalization has been stated most directly by Rao [15], who argues that the next phase of programming will shift from writing implementation code to writing formal, dependently-typed problem specifications, with AI serving as the compiler between specification and verified implementation. The two-stage pipeline Rao proposes (human intent → formal specification → verified implementation) is structurally parallel to the CBL pipeline. The key difference is domain. Rao targets general software via dependent type theory (Coq, Lean), which has no qualified toolchain for safety-critical embedded code generation. CBL targets cyber-physical systems via a synchronous state-machine semantics that maps directly onto Simulink/Stateflow and SCADE, both of which have DO-178C and ISO 26262 qualified code generators. Where Rao's intermediate representation is a dependent-typed term, CBL's is a mode-transition structure with decidable well-posedness and behavioral coverage criteria. These are properties that matter for certification arguments, not type-theoretic proof obligations.

Aerospace organizations and firms in other safety-critical sectors have developed internal workflows that address parts of this gap. Those workflows are proprietary and unavailable for independent evaluation. To our knowledge, no public-domain framework combines controlled behavioral syntax, decidable well-posedness checking, and direct compilation to mainstream MBPD toolchains.

1.4 LLMs as Stochastic Compilers

The history of software development is a history of rising abstraction, where each generation of tools enables work that was impractical at the previous level. FORTRAN replaced assembly language in 1957 by translating algebraic formulas directly into machine instructions [2]; programmers who had managed register allocation by hand accepted the trade-off because the output was efficient enough. The same bootstrapping pattern is visible in the lineage from LISP through C: each generation of tools was used to build the next, then became self-sustaining. At every transition, the dominant objection was that the new abstraction would sacrifice efficiency or render existing skills obsolete. None of these transitions eliminated programming as a discipline; each one shifted attention from lower-level mechanics to higher-level design.

Large language models continue this pattern. The specification is expressed in natural language, and the LLM translates it into executable code, much as a compiler translates algebraic formulas into machine instructions. In this framing, natural language is the new source language and the LLM is the new compiler. The analogy is structurally apt, but it conceals a critical difference. Every previous compiler in this progression was *deterministic*: given identical input, it produced identical output. The best of them were eventually verified or qualified for safety-critical use. The LLM "compiler" is stochastic. Its output on any given input is a sample from a distribution shaped by training data, prompt context, and sampling temperature. It is not verifiable in the sense that a qualified tool is verifiable, and it produces output that is plausible by construction, not correct by construction.

Three properties are required of any tool that generates code for safety-critical systems under DO-178C [16] or ISO 26262 [8]: *determinism* (identical input produces identical output), *verifiability* (the tool's correct operation can be confirmed independently of its outputs), and *qualification* (the tool has been formally credited under the applicable standard). Current large language models satisfy none of these. Their outputs are samples from a distribution, not deterministic functions of their inputs. Their internal behavior is not analyzable in the sense required by tool qualification. They cannot receive TQL-1 through TQL-4 credit under DO-178C Section 12.2 [16].

A further structural problem is circular verification. If an LLM generates both the code and the test suite for that code, and both are drawn from the same model's distribution, the tests will reflect the same systematic misinterpretations of the requirement as the code, so tests pass for reasons unrelated to correctness. The verification loop collapses into a consistency check between two artifacts produced by the same stochastic process, neither of which independently

represents the engineer's intent. This is not a failure mode that can be corrected by scaling the model; it is a consequence of using the same distribution for both tasks.

In practitioner workflows, including our own iterative development experience, a recurring failure mode is late discovery of plausible but incorrect LLM-generated code [14, 13, 1, 19, 20]. The risk compounds across four stages: debugging "almost-correct" code often costs more than writing it directly; committed defects propagate as interfaces and tests adapt around them; repeated correction cycles degrade organizational trust; and in cyber-physical systems, the consequence of a latent defect is physical. As AI-generated software increasingly mediates physical decisions, unverified outputs become systemic risk rather than routine engineering debt.

CBL's response to this problem is architectural. LLMs are restricted to the specification layer: parsing requirements, proposing structured behavioral clauses for engineer review, and generating clarification questions. They do not generate code. Code generation is delegated to the qualified MBPD toolchain (Simulink Coder, SCADE KCG), which carries certification credit that no LLM can match. Between the LLM's natural-language output and the qualified generator's input sits a deterministic well-posedness checker that enforces behavioral correctness properties independent of how the specification was produced. This architecture exploits what LLMs do well (understanding natural language, recognizing behavioral patterns across large requirement sets, conducting structured dialogue) while bypassing their structural unsuitability for certified code generation. It is not a workaround. It is the design.

1.5 Cyber-Physical Systems and the Stakes of Unreliable Code

Large language models have materially reduced the time from requirement to prototype, and this compression is beginning to reach cyber-physical development. In pure software, the opportunity translates directly into shorter iteration cycles. In systems where software commands physical actuators, the same acceleration introduces a new constraint: the cost of a behavioral error is no longer a deployment delay but a physical action applied to a real system.

The consequences of error therefore scale with the coupling between software and physics. In a web service, a defect can often be corrected in the next deployment window. In a cyber-physical system, the same class of defect can issue the wrong command to an actuator: braking at the wrong instant, injecting fuel at the wrong rate, or commanding the wrong guidance maneuver. The distinction is not one of degree but of failure mode: the correction window closes before the damage is reversible.

Cyber-physical systems operate in closed feedback loops where supervisory logic is itself a safety mechanism. In battery thermal management, flight control mode arbitration, and redundant avionics fault isolation, behavioral correctness is a safety condition, not a quality preference. If a stochastic generator produces this logic and the result is accepted without an independent verification layer, the dominant risk is not obvious failure during nominal tests. The dominant risk is latent mis-specification that remains hidden until a rare operating condition exposes it.

The acceleration of autonomy makes this issue more urgent each year. LLMs can help parse requirements, surface ambiguities, and draft candidate specifications at scale, but they are not yet suitable for certifiable code generation in cyber-physical systems with stringent assurance requirements, particularly in aerospace. The contribution LLMs make at the requirements layer is valuable, but only when bounded by a pipeline that can verify behavioral correctness independently of the generator that proposed it. Without that boundary, faster generation becomes faster propagation of subtle defects into the systems where failure carries the highest physical and societal cost.

2 What CBL Is

CBL is a *controlled behavioral language* for state-based specification. It is structured enough for formal analysis and readable enough for domain engineers. It occupies the specification layer between human intent and MBPD models.

The central design principle is verification before modeling. Completeness, determinism, coverage, and invariant consistency are checked at the specification level, in a readable document, before any model is constructed. Errors are corrected in the CBL text rather than in a graphical model. The corrected specification is then compiled, and the compiled model inherits the verified properties.

2.1 The Proposed Pipeline

CBL introduces a new step in the MBPD pipeline:

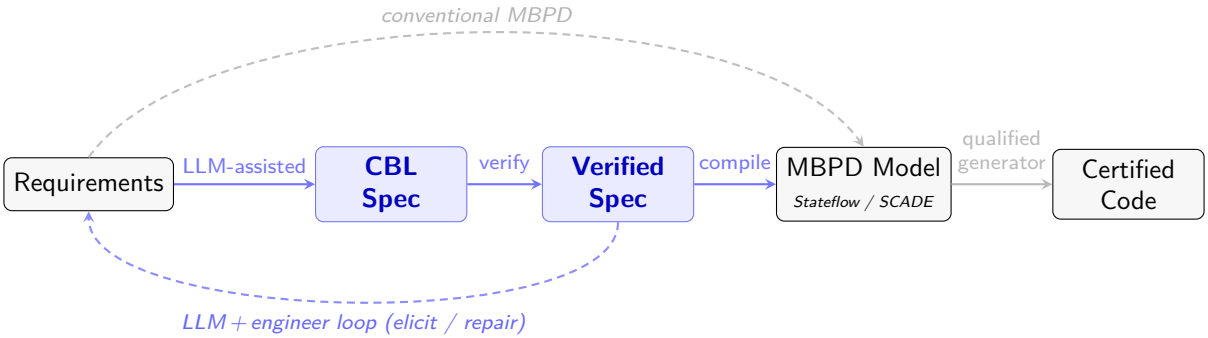

Figure 1: The CBL pipeline. Blue nodes and arrows are the new specification layer; gray arrows are existing MBPD stages.

The CBL specification is the missing verification point: a structured, checkable artifact between requirements and models. Inserting this step catches defects at their point of origin, before they propagate through the downstream pipeline.

2.2 Intellectual Heritage

CBL synthesizes proven elements from four traditions:

Source	What CBL adopts	What CBL does not adopt
EARS [12]	`shall` as commitment keyword; readability	Sentence templates only; no mode structure
SCR [6]	Mode-transition structure; decidable well-posedness	Tabular format; no NL readability
FRET [5]	Declarative timing predicates	Individual properties only; no mode structure
Contracts [3]	`Assumes/Guarantees` interface	Theoretical framework only; no syntax

Table 1: CBL's intellectual heritage: what is adopted and what is not.

2.3 Specification Structure

A CBL specification is a single `System` declaration containing ordered sections:

```
System <name>

  Assumes:
    <input declarations and environmental constraints>
```

```
5
6   Definitions:            -- optional: named predicate aliases
7     <named predicates>
8
9   Constants:              -- optional: design parameters
10    <named constants>
11
12  Guarantees:
13    <output declarations with types>
14
15  Variables:              -- optional: internal state
16    <internal state declarations>
17
18  Always:                 -- optional: global invariants
19    <proof obligations>
20
21  Initial mode: <mode_name>
22
23  Mode <mode_name>:
24    <transition rules>
```

Listing 1: CBL specification skeleton.

Assumes declares environmental constraints on inputs. In contract-based design terms, these are the preconditions. **Guarantees** declares the behavioral commitments: the output variables that every transition must assign. **Modes** are named behavioral states, each containing guarded transitions. Within each mode, transitions follow a `When...shall` pattern: the guard is a Boolean condition over inputs and state; the `shall` keyword marks the behavioral commitment, directly traceable to a corresponding `shall` in the natural-language requirement. Variables in the `Variables` section have implicit hold semantics: if a variable is not assigned by the selected `shall` block at cycle t, its value at cycle $t+1$ equals its value at cycle t.

2.4 Guards and Timing Predicates

Guards use a controlled English vocabulary of comparison predicates:

Predicate	Formal meaning
`x deviates from y and z beyond threshold`	$\lvert x-y\rvert > \tau \wedge \lvert x-z\rvert > \tau$
`x agrees with y within threshold`	$\lvert x-y\rvert \leq \tau$
`x exceeds bound`	$x > b$
`x is below bound`	$x < b$
`x equals value`	$x = v$

Table 2: Selected atomic predicates from the CBL controlled vocabulary.

Timing predicates qualify guards with persistence conditions:

```
When <guard> for N consecutive cycles,    -- fires on the Nth cycle
When <guard> for fewer than N consecutive cycles,   -- complement
```

Timing predicates are declarative: the engineer specifies the duration, and the compiler synthesizes the internal counters, increment logic, and reset conditions.

2.5 Actions

The `shall` keyword introduces the action block. Actions assign outputs, update internal state, and direct mode transitions:

Action	Meaning
`set <var> to <expr>`	Assign value
`hold <var>`	Retain previous-cycle value
`increment <var>`	var := var + 1
`reset <var>`	var := initial value
`transition to <mode>`	Next-cycle mode change
`remain in <mode>`	Self-loop

Table 3: CBL action vocabulary.

Every `shall` block must end with a transition directive and must assign all `Guarantees` variables (enforced by the well-posedness checker). Entry action semantics are formalized in Section 5.

3 A First Example: Three-Phase Traffic Signal

Before defining formal semantics and design principles, a concrete example establishes the syntax and working vocabulary. A traffic signal controller requires no domain background: the reader can verify correctness by inspection. Two requirements define the system:

> **REQ-1:** The signal shall cycle through green, yellow, and red phases in order, returning to green after each red phase.
>
> **REQ-2:** Each phase shall hold for a configurable duration before advancing: 45 cycles for green, 5 for yellow, 60 for red.

```
System TrafficSignal

  Constants:
    RED_CYCLES    : integer [1..inf) = 60
    GREEN_CYCLES  : integer [1..inf) = 45
    YELLOW_CYCLES : integer [1..inf) = 5

  Guarantees:
    light : {red, green, yellow}

  Initial mode: red

  Mode red:
    When true for RED_CYCLES consecutive cycles,
    shall set light to green,
         transition to green.
    Otherwise,
    shall set light to red,
         remain in red.

  Mode green:
    When true for GREEN_CYCLES consecutive cycles,
    shall set light to yellow,
         transition to yellow.
```

```
    Otherwise,
    shall set light to green,
         remain in green.

  Mode yellow:
    When true for YELLOW_CYCLES consecutive cycles,
    shall set light to red,
         transition to red.
    Otherwise,
    shall set light to yellow,
         remain in yellow.
```

Listing 2: A three-phase traffic signal in CBL.

Each mode has exactly two guards: a timing predicate (`true for N consecutive cycles`) and `Otherwise`. The pair is exhaustive and mutually exclusive. The timing predicates enforce REQ-2: each phase holds for its configured duration, after which the controller advances to the next phase. The CBL compiler synthesizes the necessary counter logic in the generated Stateflow model. The worked examples (Section 8) extend this specification with pedestrian crossing and emergency preemption.

4 Scope and Exclusions

CBL is defined as much by what it excludes as by what it includes.

1. **CBL is not a programming language.** CBL is a behavioral contract: a description of *what* the system does, not *how* it does it. It has no loops, no memory allocation, no function calls, no I/O operations. The transition system provides the formal object against which completeness, determinism, and coverage are defined.

2. **CBL is not a replacement for Simulink, Stateflow, or SCADE.** CBL is a front-end that produces verified behavioral specifications compilable to these tools. The output of CBL compilation is a model within the engineer's existing workflow, not an artifact requiring unfamiliar tooling. Engineers inspect, simulate, and modify the compiled model using their standard tools.

3. **CBL is not a formal methods language.** CBL is deliberately less expressive than temporal logic. It does not support the full expressiveness of LTL or CTL. Its scope is state-based behavioral specification with guards over discrete and threshold-based conditions. The restricted expressiveness is what makes well-posedness decidable and compilation structure-preserving.

4. **CBL is not a requirements management tool.** It does not replace DOORS, Polarion, or Jama. CBL specifications are derived from requirements and trace back to them, but CBL does not manage the requirements lifecycle.

5. **CBL is not a code generator.** CBL compiles to models, not to code. Code generation is delegated to qualified MBPD tools (Simulink Coder, SCADE KCG). This is a deliberate architectural decision that preserves the certified trust chain.

6. **CBL is not YAML, JSON, or SysML.** CBL uses controlled English syntax, readable by domain engineers without formal-methods training. The concrete syntax is designed for human readability and traceability to natural-language requirements, not for machine interchange.

7. **CBL does not target numerical algorithms, continuous control laws, or signal processing.** The scope boundary is defined by the class of correctness criteria CBL can express: properties of a finite-state transition system with guards over discrete and threshold-based conditions. Mode management, fault handling, voting, and supervisory control are within scope. Kalman filters, PID controllers, and FFTs are not.

5 Formal Semantics

A CBL specification induces a synchronous Mealy machine over modes (S), inputs (U, from `Assumes`), internal state (X, from `Variables` plus compiler-synthesized counters), and outputs (Y, from `Guarantees`). A transition function and an output function are evaluated at each discrete step. CBL targets the synchronous execution semantics of Stateflow and SCADE. Asynchronous, event-driven behavior is outside the current scope. An `Otherwise` clause is syntactic sugar for the negation of all preceding guards in the mode.

5.1 Timing Predicate Expansion

The compiler transforms declarative timing predicates into counter-augmented guards. For a guard G in mode s with predicate `for` N `consecutive cycles`:

1. Synthesize an internal counter initialized to zero.
2. On each step in mode s: if G is true, increment the counter; otherwise reset to zero.
3. On entry to mode s, reset the counter to zero.
4. Replace the timed guard with: $G \wedge (\text{counter} \geq N)$.
5. The complement (`for fewer than` N `consecutive cycles`) becomes: $G \wedge (\text{counter} < N)$.

After expansion, all guards are pure Boolean expressions over inputs and internal state, and the standard well-posedness conditions apply.

5.2 Entry Actions and Hold Semantics

Entry actions execute on the first cycle after a mode transition, before guard evaluation in the entered mode. They do not fire on self-loops. The initial mode's entry actions execute at startup. Entry actions can assign internal variables and outputs, and their assignments count toward action totality on the entry cycle.

The `hold` action retains the previous-cycle value of a guaranteed output. In Stateflow, this corresponds to omitting the assignment (Stateflow data naturally retains its value). In SCADE, it maps to the `last` operator. Variables in the `Variables` section have implicit hold semantics: if a variable is not assigned by the selected `shall` block, its value persists.

5.3 Well-Posedness

Definition 1 (Well-Posedness)**.** *A CBL specification is* well-posed *if and only if:*

1. ***Guard exclusivity:*** *In each mode, at most one guard evaluates to true for any input and internal state.*
2. ***Guard completeness:*** *In each mode, at least one guard evaluates to true for every reachable input and internal state (explicit* `Otherwise` *permitted).*
3. ***Action totality:*** *Every transition assigns all guaranteed outputs and all updated internal variables.*
4. ***Invariant consistency:*** *Mode invariants are satisfiable and preserved by all transitions entering the mode.*

The prototype checker enforces guard exclusivity structurally, by detecting provable guard overlaps from syntactic patterns; guard completeness by requiring an explicit `Otherwise` *clause or issuing a*

warning; action totality fully; and invariant consistency partially, through range-bound validation. Full semantic analysis of guard exclusivity, guard completeness, and invariant consistency requires SMT integration (Section 10).

Well-posedness implies that the specification defines a deterministic, total transition system [7, 18]. The check occurs at the specification level, before any model exists. Conflicting transitions, unguarded paths, and unassigned outputs are found and corrected in a readable text specification rather than in a graphical model.

5.4 Behavioral Coverage at the Specification Level

DO-178C Level A requires Modified Condition/Decision Coverage (MC/DC) for code-level testing. CBL applies the same coverage discipline at the specification level, before any model or code exists.

Three coverage criteria are defined over the transition system induced by the specification:

1. **State coverage:** Every mode $s \in S$ is visited by at least one trace.
2. **Transition coverage:** Every guard in every mode is exercised (evaluates to true) in at least one trace.
3. **Guard condition independence:** For every compound guard with atomic conditions $c_1, \ldots, c_k$, each c_i independently affects the guard outcome in at least one trace. This is MC/DC applied to specification guards rather than code branches.

Coverage gaps (unreachable modes, redundant guard conditions, transitions that cannot fire) are detected before modeling begins. These gaps indicate either specification errors (a mode that should be reachable but is not) or requirement gaps (a condition that is always subsumed by another). In both cases, the defect is corrected in the readable CBL text, not in a graphical model.

The reasoning engine (`cblc reason`) contributes to coverage analysis through rule R12 (reachability: warns if a mode is unreachable from the initial mode via any sequence of transitions) and rule R13 (unused declarations: warns about variables or constants that are declared but never referenced).

5.5 Compilation Correctness: Trace Equivalence

The correctness criterion for CBL-to-Stateflow and CBL-to-SCADE compilation is *trace equivalence*: for every finite input sequence, the CBL specification and the compiled model produce identical output sequences and identical mode sequences.

CBL compiles to a restricted fragment of Stateflow that excludes event broadcasting, inter-chart communication, super-step semantics, early return logic, MATLAB functions inside states, history junctions, and Stateflow temporal operators (`after`, `before`, `at`). Within this fragment, the one-to-one structural mapping (Table 5) preserves the Mealy machine abstraction.

Trace equivalence is validated empirically in the current prototype: the CBL source is compiled to both a Python reference backend and a Stateflow chart; both are driven with identical input sequences, and the output traces are compared cycle by cycle. The traffic signal examples pass this round-trip validation. A formal proof of trace equivalence via structural induction over the compilation mapping is planned as future work.

6 Design Principles

6.1 The LLM Boundary

The role of large language models in the CBL pipeline is precise and bounded:

LLMs are used where they perform well (natural-language understanding, pattern recognition) and are excluded from tasks requiring deterministic, certifiable code generation. This is not a

What the LLM does	What the LLM does not do
Parse natural-language requirements	Generate code
Propose CBL clauses with confidence scores	Perform verification
Identify ambiguities in requirements	Make design decisions
Generate clarification questions	Replace engineer judgment

Table 4: The LLM boundary in the CBL pipeline.

limitation; it is a design principle. By restricting LLMs to the specification layer and routing all implementation through qualified toolchains, the framework preserves the MBPD trust chain. LLM errors are caught at the specification level (well-posedness checks, engineer review) before they reach the qualified pipeline.

The three-layer pipeline is a domain-specific instantiation of the neuro-symbolic integration principle: the LLM provides natural-language understanding and pattern recognition; the reasoning engine (`cblc reason`) and CBL compiler provide formal inference and acceptance control. Recent results in the AI reasoning literature confirm that delegating inference to a symbolic engine rather than asking an LLM to reason in natural language yields significant accuracy improvements, particularly as reasoning depth increases [4]. In CBL the separation carries additional weight: the symbolic gates enforce the well-posedness conditions that support the DO-178C and ISO 26262 certification argument, so they cannot be bypassed regardless of the neural component's confidence.

6.2 Interactive Elicitation and Refinement

CBL supports incremental, partial specification with systematic refinement. The LLM-assisted elicitation loop follows five steps:

1. The LLM parses natural-language requirements into an initial (incomplete) CBL specification.
2. The well-posedness checker identifies deficiencies: missing transitions, overlapping guards, unassigned outputs.
3. For each deficiency, the LLM generates a targeted clarification question for the engineer.
4. The engineer answers; the LLM proposes a refined specification.
5. Iterate until the specification passes all well-posedness checks.

The LLM proposes; the checker verifies; the engineer decides. The loop is human-in-the-loop by construction.

6.3 Structure-Preserving Compilation

CBL compiles to Stateflow and SCADE via one-to-one structural mappings:

CBL Element	Stateflow	SCADE
Mode $s \in S$	State	SSM state
Guard G	Transition condition `[G]`	SSM transition condition
`shall` assignment	Transition action	State action / output equation
Entry action	`en:` action	State action
Timing predicate	Local counter + guard	Local counter + guard
Internal variable	Local data	Local flow
Hierarchical mode (planned)	Superstate	Nested state machine
`Otherwise`	Default transition	Default transition

Table 5: Structure-preserving compilation mappings.

The generated model is not an opaque artifact; it is a readable model that engineers can inspect, simulate, and modify in their standard tools. Well-posedness of the CBL source guarantees that the compiled model has no conflicting transitions, no incomplete guard coverage, and no unintended default behaviors.

6.4 The Tool Qualification Argument

The CBL compiler is an unqualified development tool. Under DO-178C Section 12.2, tool qualification level depends on whether tool errors are detectable by downstream verification. The compiler's output is a *model*, not code. That model enters the standard MBPD verification process: model review, MIL testing, Simulink Design Verifier analysis, SIL/HIL testing.

This places the compiler at **TQL-5** (DO-178C [16]) or **Tool Confidence Level 1** (ISO 26262 Part 8 [8]): a tool whose errors can be detected by subsequent activities. The qualified trust chain runs from the verified model through the qualified code generator to certified code. The CBL compiler sits upstream, and its outputs are checked before entering that chain.

6.5 CBL as a Platform in Platform-Based Design

In the platform-based design lineage, CBL occupies the articulation point between requirements and models. That lineage runs from the Y-chart formulation of Gajski and Kuhn through the embedded-systems elaboration of Sangiovanni-Vincentelli and collaborators [10, 17]. CBL is the *specification platform*: a family of behavioral descriptions from which refinement proceeds to the *modeling platform* (Simulink/Stateflow, SCADE); the CBL compilers implement that refinement mapping. This mapping is structure-preserving, and its output is subject to the same verification that any model receives before entering the qualified code generation step.

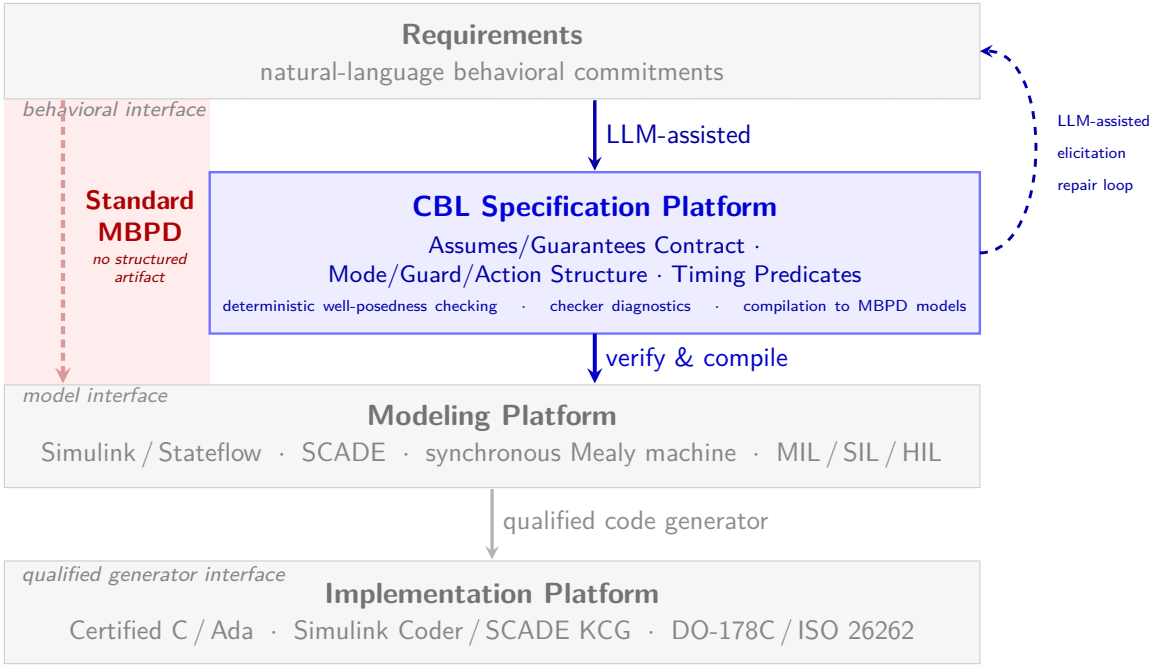

Figure 2: CBL in the Sangiovanni-Vincentelli platform-based design hierarchy [10, 17]. Each horizontal band is a *platform*: a library of behavioral or structural abstractions with a well-defined interface to the adjacent level. Standard MBPD (dashed) crosses the requirements-to-model interface without a structured artifact, allowing requirement defects to propagate unchecked. CBL introduces a specification platform at that interface: a verifiable, engineer-readable artifact produced by LLM-assisted elicitation and checked for well-posedness before any model is constructed.

6.6 Traceability

The CBL pipeline produces a traceability chain from natural-language requirements through CBL clauses, IR elements, model artifacts, and generated code. Each link in this chain is explicit and auditable. The requirements-to-CBL mapping is established during elicitation (the traceability mitigation module verifies bidirectional coverage). The CBL-to-IR mapping is defined by the compilation semantics. The IR-to-model mapping is defined by the Stateflow or SCADE code generator. The model-to-code mapping is handled by the qualified MBPD code generator.

DO-178C mandates a requirements-to-design traceability matrix as a certification artifact. CBL automates the first two links: every CBL clause traces to a requirement (by the elicitation process), and every Stateflow state traces to a CBL clause (by construction). The remaining manual effort reduces to verifying the initial requirements-to-CBL mapping, which is the point where engineer judgment is both necessary and most effective.

7 Trust Architecture

7.1 Trust Zones and Validation Gates

The CBL pipeline partitions components into six trust zones, ordered by the strength of their correctness guarantees:

Trust Zone	Components	Verification Basis
Untrusted	LLM (extract, revise)	Statistical model; may hallucinate
Mitigated	6 hallucination mitigation modules	Deterministic checks; no LLM in checking path
Verified	Reasoning engine (`cblc reason`, OCaml; 13 rules)	Rule-based; same input → same output
Qualified	CBL compiler (`cblc`, OCaml)	Type-safe; sole acceptance oracle
Certified	MBPD toolchain (Simulink Coder, SCADE KCG)	Vendor-supplied qualification kits
Authoritative	Engineer + provenance audit log	Human judgment; append-only record

Table 6: Trust zones in the CBL pipeline, from least trusted to most authoritative.

Three validation gates enforce trust boundaries between zones. Each gate is deterministic Python code within the session orchestrator.

1. **Schema Gate** (LLM → Reasoner): Validates `extracted_facts.json` against a published JSON Schema. Retries the LLM up to three times on schema violations; aborts if the output remains invalid.
2. **Provenance Gate** (LLM → Reasoner): Unconditional. The orchestrator overwrites every provenance tag the LLM assigns: facts matching engineer text become `user_stated`, previously confirmed facts become `user_confirmed`, and everything else is forced to `llm_inferred`. The LLM cannot influence its own trust level.
3. **Verdict Gate** (`cblc reason` → `cblc ingest`): Validates the reasoning verdict against a verdict schema. Fatal on failure: a malformed verdict halts the iteration immediately rather than propagating to the compiler.

No untrusted data reaches the CBL compiler without passing all three gates. The architecture is defense-in-depth: each trust boundary is enforced by a different mechanism (schema validation, provenance rewriting, verdict validation), so a failure at one gate does not compromise the gates downstream.

7.2 Provenance as Epistemic Tracking

Every semantic value in the extracted-facts representation carries a provenance tag that records its epistemic status: how the value entered the pipeline and whether a human has confirmed it. Seven tags form an ordered commitment hierarchy:

Tag	Meaning	Assigned By
`user_stated`	Engineer wrote this in original requirements	Orchestrator
`user_confirmed`	Engineer explicitly approved	Orchestrator
`default_assumed`	System default (e.g., initial values)	Orchestrator
`llm_inferred`	LLM proposed; not yet confirmed	Orchestrator
`rule_derived`	Reasoner derived from confirmed premises	Reasoning engine
`rule_derived_pending`	Reasoner derived from unconfirmed premises	Reasoning engine
`user_rejected`	Engineer rejected; permanently excluded	Orchestrator

Table 7: Provenance tags and their assignment sources.

The commitment policy is strict: only facts tagged `user_stated`, `user_confirmed`, `rule_derived`, or `default_assumed` may enter the final specification. Facts tagged `llm_inferred` block compilation (reasoning rule R9) until the engineer promotes them via explicit confirmation. Rejected facts are permanently excluded and never re-proposed.

Provenance propagates transitively through the reasoning engine. A `rule_derived` fact inherits the weakest provenance among its premises: if any premise is `llm_inferred`, the derived fact receives `rule_derived_pending` and requires confirmation before commitment. This prevents an unconfirmed LLM inference from silently entering the specification through a chain of otherwise-valid deductions.

An append-only audit log records every provenance assignment, confirmation, and rejection, providing a complete record for certification audits (DO-178C traceability matrix, ISO 26262 safety case).

7.3 LLM Hallucination Mitigations

Six post-success mitigation modules run after the specification passes the well-posedness checker. All are deterministic: no LLM participates in the checking path. Their role is to surface latent hallucinations that are structurally valid (they passed the checker) but semantically wrong (they do not reflect the engineer's intent).

Module	Description
Back-translation	Renders the CBL specification back to plain English for side-by-side comparison with the original requirements. Discrepancies surface semantic drift.
Dual extraction	Runs LLM extraction twice with different prompts and structurally diffs the results. Divergences indicate instability and hallucination risk.
Mutation testing	Generates AST-level mutants (flipped guards, swapped transition targets, dropped assignments) and checks whether the test suite detects each mutation.
Pattern justification	Validates that every domain-pattern recommendation cites a real requirement with plausible lexical overlap. Catches spurious pattern insertions.
Provenance control	Deterministic provenance assignment; maintains the append-only audit log described in Section 7.2.
Traceability	Verifies bidirectional coverage: every fact cites a real requirement fragment (forward), and every requirement is addressed by at least one fact (backward, surjectivity).

Table 8: LLM hallucination mitigation modules. All are deterministic; no LLM participates in the checking path.

A separate Z3 guard checker, integrated into the CBL compiler rather than the Python

mitigation layer, uses SMT queries for semantic guard exclusivity ($\text{SAT}(G_i \wedge G_j) = \text{false}$) and completeness ($\text{SAT}(\neg(G_1 \vee \cdots \vee G_n)) = \text{false}$), strengthening the reasoning engine's heuristic analysis (rules R2 and R10).

The mitigations complement the well-posedness checker. The checker enforces structural correctness (the specification is a valid transition system). The mitigations enforce semantic fidelity (the specification reflects what the engineer intended). A specification that passes both has been checked for well-posedness, provenance integrity, requirement coverage, and behavioral stability under re-extraction.

8 Worked Examples

Two extensions build on the basic traffic signal introduced in Section 3. Each adds modes, inputs, and outputs to the previous specification; the domain requires no additional background. All specifications are parsed, checked for well-posedness, and compiled to Stateflow charts by the prototype toolchain.

8.1 Pedestrian Crossing Extension

8.1.1 Additional Requirements

REQ-3: When a pedestrian crossing request is active at the end of a green phase, the signal shall service the request through a dedicated walk phase before resuming the green phase.

REQ-4: During the pedestrian phase, the walk signal shall be active and the vehicle light shall remain red for the configured pedestrian duration (20 cycles).

8.1.2 CBL Extension

Two new modes (`ped_waiting`, `ped_crossing`), one input (`ped_request`), one output (`walk_signal`), and one constant (`PED_CYCLES`) suffice. The `green` mode is the only existing mode that changes structurally; the remaining modes gain a `walk_signal` assignment to satisfy action totality.

```
Constants:
  PED_CYCLES : integer [1..inf) = 20

Assumes:
  ped_request is a boolean signal

Guarantees:
  walk_signal : {walk, dont_walk}

Mode green:
  When true for GREEN_CYCLES consecutive cycles
   and ped_request is true,
  shall set light to yellow,
       set walk_signal to dont_walk,
       transition to ped_waiting.
  When true for GREEN_CYCLES consecutive cycles
   and not ped_request is true,
  shall set light to yellow,
       set walk_signal to dont_walk,
       transition to yellow.
  Otherwise,
  shall set light to green,
       set walk_signal to dont_walk,
```

```
        remain in green.

  Mode ped_waiting:
    When true for YELLOW_CYCLES consecutive cycles,
    shall set light to red,
         set walk_signal to walk,
         transition to ped_crossing.
    Otherwise,
    shall set light to yellow,
         set walk_signal to dont_walk,
         remain in ped_waiting.

  Mode ped_crossing:
    When true for PED_CYCLES consecutive cycles,
    shall set light to green,
         set walk_signal to dont_walk,
         transition to green.
    Otherwise,
    shall set light to red,
         set walk_signal to walk,
         remain in ped_crossing.
```

Listing 3: Pedestrian extension: modified `green` mode and new modes (excerpt).

The `green` mode has three guards: two timing predicates conditioned on pedestrian demand, and `Otherwise`. The pair of conditional timing guards partitions the green-phase-end event by pedestrian request state; the well-posedness checker confirms their mutual exclusivity. Pedestrian requests are sampled at the end of the green phase (cycle 45), a policy encoded directly in the guard structure. The `PED_CYCLES` constant enforces REQ-4's 20-cycle pedestrian crossing duration.

8.2 Emergency Preemption and Fault Handling

8.2.1 Additional Requirements

REQ-5: An emergency preemption signal shall immediately override all normal and pedestrian modes and hold all vehicle lights at red until the preemption clears, at which point the signal shall return to the red phase.

REQ-6: If a lamp fault is detected and no emergency is active, the system shall enter a fault-flash mode, assert the fault flag, and inhibit the walk signal until the fault clears.

8.2.2 CBL Extension

Two inputs (`emergency`, `lamp_fault`), two new modes (`emergency_hold`, `fault_flash`), and one output (`fault_flag`) are added. A `Definitions` clause names the nominal-operation precondition shared across all guards:

```
  Assumes:
    emergency is a boolean signal
    lamp_fault is a boolean signal

  Guarantees:
    fault_flag : boolean

  Definitions:
```

```
    normal_operation means not emergency is true
                      and not lamp_fault is true

  Mode green:
    When emergency is true,
    shall set light to all_red,
         set walk_signal to inhibited,
         set fault_flag to false,
         transition to emergency_hold.
    When lamp_fault is true and not emergency is true,
    shall set light to dark,
         set walk_signal to inhibited,
         set fault_flag to true,
         transition to fault_flash.
    When true for GREEN_CYCLES consecutive cycles
     and ped_request is true and normal_operation,
    shall set light to yellow,
         set walk_signal to dont_walk,
         set fault_flag to false,
         transition to ped_waiting.
    When true for GREEN_CYCLES consecutive cycles
     and not ped_request is true and normal_operation,
    shall set light to yellow,
         set walk_signal to dont_walk,
         set fault_flag to false,
         transition to yellow.
    Otherwise,
    shall set light to green,
         set walk_signal to dont_walk,
         set fault_flag to false,
         remain in green.

  Mode emergency_hold:
    When not emergency is true,
    shall set light to red,
         set walk_signal to dont_walk,
         set fault_flag to false,
         transition to red.
    Otherwise,
    shall set light to all_red,
         set walk_signal to inhibited,
         set fault_flag to false,
         remain in emergency_hold.
```

Listing 4: Supervisor extension: priority definition and `green` mode (excerpt).

Priority is encoded structurally: guard evaluation order within a mode determines precedence, with no additional syntax required. The `normal_operation` definition eliminates repeated negation without introducing new language constructs. Table 9 summarizes the specification growth across the three examples.

Example	Modes	Inputs	Outputs	Transitions	Lines
Basic signal	3	0	1	6	35
With pedestrian	5	1	2	11	87
Full supervisor	7	3	3	26	175

Table 9: Specification growth across the traffic signal examples.

9 The Prototype Toolchain

The CBL compiler (`cblc`), implemented in OCaml, provides the production front end for the CBL toolchain, including a three-layer NLP-to-CBL pipeline for LLM-assisted specification. Figure 3 shows the system architecture.

9.1 CBL Compiler and Three-Layer NLP Pipeline

The compiler implements the parser, checker, and code generator in a statically typed language suitable for production use. It also provides an `ingest` command that accepts structured JSON facts (the output of the NLP pipeline) and constructs a well-posed CBL specification from them.

The NLP-to-CBL pipeline separates concerns across three layers spanning two implementation languages aligned with the trust boundary. The untrusted extraction layer is an LLM driven by a Python orchestrator; the two deterministic layers (reasoning and CBL construction) are implemented in OCaml as subcommands of a single `cblc` binary:

Layer 1 (LLM → extracted facts). An LLM parses natural-language requirements into a typed JSON fact set conforming to a published schema. Every fact carries a provenance tag (`user_stated`, `llm_inferred`, `rule_derived`, `default_assumed`) that tracks its epistemic status. The LLM never emits CBL text; it emits structured data.

Layer 2 (reasoning engine). The `cblc reason` subcommand (OCaml) loads the extracted facts and applies 13 consistency rules: type compatibility, guard completeness, action totality, transition target validity, and duplicate detection. It reuses the compiler's Z3-backed well-posedness checker for the structural and type rules and adds the provenance and structural-completeness rules. For each violation, the engine proposes a deterministic repair and generates a clarification question for the engineer. Provenances propagate: a rule-derived fact inherits the weakest provenance of its premises, ensuring that conclusions drawn from unconfirmed premises require confirmation before entering the final specification. The engine emits a verdict (pass/fail with committed and pending fact partitions).

Layer 3 (CBL construction). The `cblc ingest` command reads the committed facts from the reasoning verdict, constructs a typed AST, runs the well-posedness checker, and emits the final `.cbl` file. This is the sole acceptance oracle: a specification is accepted only when the compiler returns zero errors.

A Python session orchestrator manages the iteration protocol. If the reasoning engine or well-posedness checker reports errors, the orchestrator feeds diagnostics back to Layer 1 for refinement, continuing until the specification passes or a stall is detected (two consecutive iterations with no progress). The full pipeline has been validated end-to-end: a set of extracted facts for a traffic-light controller produces a 35-line well-posed CBL specification in a single iteration with zero warnings.

The principle of delegating formal inference to an external symbolic engine rather than asking an LLM to reason directly is independently validated in the logical reasoning literature. Di et al. [4] propose LoRP, a framework that uses an LLM to translate natural-language queries into Prolog and delegates deductive inference to SWI-Prolog; empirical results show significant accuracy gains over pure-LLM reasoning, particularly as inference depth increases. CBL's Layer 2 applies the same separation of concerns—a deterministic symbolic engine (`cblc reason`), not the LLM, performs the inference—to specification consistency rather than query answering: the 13 rules enforce well-posedness, and the engine's verdict controls admission to Layer 3 regardless of how Layer 1 produced the extracted facts.

9.2 Reasoning Rules

The 13 consistency rules in the reasoning engine (`cblc reason`) partition into errors (which block compilation) and warnings (which flag issues for review). The structural, type, and reachability

rules reuse the compiler's Z3-backed checker (`checker.ml`); the empty-mode, no-initial-mode, unused-declaration, and provenance rules (R1, R5b, R13, R8, R9, R9b) are implemented in `reason.ml`.

Rule	Name	Description
R1	Empty mode	Every mode must have at least one transition.
R2	Guard completeness	Every mode needs an `Otherwise` clause (syntactic totality).
R3	Action totality	Every transition must assign all guarantees that lack a default.
R4	Invalid target	Transition target must reference a declared mode.
R5	Invalid initial	The declared initial mode must exist.
R5b	No initial mode	At least one initial mode must be declared.
R6	Duplicate declarations	No name may appear in more than one namespace.
R7	Undeclared reference	Every name in guards, actions, predicates, or entry actions must be declared.
R8	Unknown constant	Constants with value `__unknown__` block compilation.
R9	Unconfirmed inference	Facts with `llm_inferred` provenance block compilation until confirmed.
R9b	Unconfirmed default	Guarantee defaults with `llm_inferred` provenance are flagged.
R10	Guard exclusivity	Heuristic warning when two guards in the same mode are not syntactically exclusive.
R11	Type checking	Inferred type of a `set` action must be compatible with the guarantee's declared type.
R12	Reachability	Warn if a mode is unreachable from the initial mode (tabled transitive closure).
R13	Unused declaration	Warn if a declared name is never referenced.

Table 10: Reasoning engine (`cblc reason`): 13 consistency rules. R1–R9 are errors that block compilation; R10–R13 are warnings.

Rules R1–R4 enforce structural well-posedness. R5–R8 catch declaration-level errors. R9 and R9b enforce the provenance commitment policy: no unconfirmed LLM inference enters the final specification. R10–R13 provide diagnostic feedback that improves specification quality without blocking compilation.

9.3 Iteration Protocol

A complete iteration proceeds through ten steps:

1. **Extract:** The LLM produces `extracted_facts.json` from natural-language requirements (iteration 1) or revises based on prior diagnostics (iterations 2+).
2. **Schema validate:** Python `jsonschema` enforces structure. Up to three retries on failure; abort if exhausted.
3. **Provenance enforce:** The orchestrator overwrites all provenance tags. The audit log is appended.
4. **Reason** (`cblc reason`): The 13 consistency rules fire. Repairs and questions are generated. Facts are partitioned into committed and pending sets.
5. **Verdict validate:** The verdict schema is enforced. Fatal on failure.
6. **Engineer interact:** Diagnostics, questions, and repair proposals are presented. The engineer confirms or rejects.
7. **Compile** (on pass): The well-posedness checker runs. If errors remain, diagnostics feed back into the next iteration.
8. **Mitigations:** Back-translation, traceability, Z3 guard analysis, mutation testing, and

pattern justification run on the accepted specification.

9. **Stall detection:** If two consecutive iterations show no improvement, the engineer is offered manual resolution or abort.
10. **Termination:** Success when the compiler passes with zero errors, or the maximum iteration count (default 10) is exhausted.

9.4 Inter-Layer Data Formats

All communication between layers uses JSON with explicit schemas:

Artifact	Producer	Consumer	Schema
`extracted_facts_N.json`	LLM (Layer 1)	`cblc reason` (Layer 2)	Published JSON Schema
`verdict_N.json`	`cblc reason` (Layer 2)	`cblc ingest` (Layer 3)	Published JSON Schema
`spec.cbl`	Compiler (Layer 3)	Engineer / MBPD	CBL grammar (EBNF)
`provenance_audit.json`	Orchestrator	Auditor	Append-only log

Table 11: Inter-layer data artifacts, their producers, consumers, and governing schemas.

Every semantic value in `extracted_facts.json` is wrapped as a `{value, provenance}` pair. The pre-enforcement JSON is preserved in the working directory before provenance rewriting, enabling post-hoc audit of raw LLM output.

10 Future Plans

10.1 Language Extensions

Several language extensions are planned for subsequent versions of CBL:

Parameterized modes. Mode templates instantiated with different parameters (e.g., a generic "sensor isolated" mode parameterized by which sensor is isolated), reducing specification duplication for symmetric systems.

Composition syntax. Explicit syntax for composing CBL specifications of multiple components, with interface contracts checked at composition boundaries. This connects CBL to contract-based design verification tools.

History semantics. Re-entry to a hierarchical mode that resumes from the last active submode, matching Stateflow's history junction behavior.

Array and vector types. Support for specifying behavior over indexed collections of sensors, actuators, or channels, enabling CBL to scale to systems with configurable redundancy levels.

10.2 Temporal Extensions

Augment CBL with richer temporal semantics: dwell-time constraints, timeouts, deadlines, and connections to temporal logic operators (**G**, **F**, **U**). This enables CBL to express liveness properties and to interface with runtime verification frameworks.

10.3 Hybrid System Integration

Extend the intermediate representation to capture the interface between CBL behavioral logic and continuous dynamics: continuous state, per-mode dynamics, discrete transitions, and guard predicates over the product space. This connects CBL to hybrid system verification tools and enables reasoning about safety across mode boundaries.

10.4 SCADE Backend

The Stateflow backend is implemented and validated. A SCADE backend targeting SCADE State Machines (SSM) is the next compilation target, providing full coverage of the two dominant MBPD toolchains. SCADE's synchronous semantics align naturally with CBL's execution model.

10.5 Semantic Guard Analysis

The Z3-based guard checker (Section 7.3) already performs semantic exclusivity and completeness queries for individual modes. For industrial deployment, the SMT integration requires hardening: full coverage of compound guards with arithmetic sub-expressions, compositional analysis across mode boundaries, and performance validation on specifications with hundreds of modes.

10.6 Scalability and Industrial Validation

Evaluate the framework on industrial-scale requirement sets with thousands of requirements. Conduct controlled deployment in an industrial MBPD workflow, measuring the impact of CBL on development time, defect rates, and certification effort across a full development cycle.

10.7 Experimental Evaluation Design

A controlled experiment is planned to evaluate CBL's impact on specification quality. Three groups perform identical behavioral specification tasks (triple-redundant sensor FDI, chosen because it exercises compound guards, timing conditions, and nested fault escalation):

Group A (Manual): Engineer translates requirements directly to Stateflow (standard MBPD practice).

Group B (CBL + LLM): Engineer uses the LLM-assisted CBL pipeline to produce a verified specification, then compiles to Stateflow.

Group C (CBL only): Engineer hand-writes a CBL specification (no LLM), verifies it, and compiles to Stateflow. This group isolates the value of CBL as a language from the value of LLM-assisted elicitation.

Four metrics are measured: (1) specification *completeness* (fraction of requirement-implied behaviors present in the final model, measured against a reference model); (2) *defect rate* (specification defects per requirement, classified by type: missing transition, inconsistent guard, unhandled case); (3) *time* (total engineer time to verified model, including CBL overhead); (4) *downstream rework* (defects discovered during model-level testing that trace to the requirements-to-model step). The experiment is single-domain; results establish proof of concept, not broad generalizability.

10.8 Integration with Contract-Based Design Tools

Connect CBL contracts to contract-based design frameworks (e.g., OCRA) to enable compositional verification: verify system-level properties from component-level CBL contracts without constructing a monolithic model.

10.9 Benchmark Suite

Construct a public benchmark of behavioral specification tasks for safety-critical systems: FDI logic, mode management, redundancy management, degraded-mode operation. Provide reference CBL specifications, reference models, and coverage-complete trace sets to enable reproducible comparison of specification methods.

10.10 Agentic Specification Framework

The three-layer NLP pipeline (Section 9.1) is orchestrated by a structured multi-agent framework in which each agent operates in a narrow, verifiable scope and the well-posedness checker remains the single point of formal verification. Six agents and one coordinating skill are implemented; additional domain agents and cross-cutting agents are planned.

10.10.1 Pipeline Agents

Five domain-agnostic agents correspond to stages of the specification workflow:

Requirements analyst. Structures raw natural-language requirements: identifies inputs, outputs, modes, timing constraints, and fault conditions. Disambiguates underspecified requirements by generating targeted clarification questions. Outputs a structured requirement set ready for translation.

CBL specifier. The core translation agent. Takes structured requirements and drives the three-layer pipeline (LLM extraction, OCaml reasoning, compiler construction) until the specification passes well-posedness checks. Wraps the session orchestrator and manages the iteration loop.

Verification engineer. Interprets checker diagnostics (REF-1 . . . 5, TYPE-1 . . . 2, WP-1 . . . 4) in terms the domain engineer understands. Proposes targeted fixes and distinguishes specification errors from missing requirements. Flags cascading fixes proactively.

Test engineer. Generates concrete test scenarios (input sequences per cycle) covering structural, timing boundary, and fault injection cases. Executes the Python reference backend and validates traces against the original requirements. Reports coverage gaps: modes never entered, guards never triggered, outputs never exercised.

Integration engineer. Compiles the well-posed CBL specification to Stateflow or SCADE via the JSON IR, runs the round-trip validation pipeline (Python traces vs. MATLAB traces), and reports any semantic discrepancies between backends.

A coordinating skill (*specify-system*) orchestrates the full pipeline: requirements analyst → domain agent → CBL specifier → verification engineer, triggered as a single workflow.

10.10.2 Domain Agents

The framework supports user-defined domain agents that carry discipline-specific vocabulary mappings, canonical specification patterns, completeness checklists, and certification awareness for the applicable standard. Each domain agent follows the same structure: it maps domain terminology to CBL constructs, provides reference patterns as starting points, and flags common omissions. Examples of domains suited to domain agents include fault detection and isolation (FDI), automotive mode management (ISO 26262), medical device alarm escalation (IEC 62304), industrial safety interlocks (IEC 61508/61511), and robotic mission management. An FDI domain agent has been implemented as the first instance, mapping terms such as "sensor deviation" to `deviates from ... beyond threshold` and carrying a complete triple-sensor FDI reference specification.

10.10.3 Cross-Cutting Agents

The framework also supports cross-cutting agents that address concerns spanning multiple components. A safety analyst agent, for example, could derive fault modes from FMEA or FTA analysis and translate each identified hazard into CBL fault-handling requirements. A composition agent could manage multi-component specifications and verify that component-level guarantees satisfy system-level assumes at composition boundaries. These agents are not yet implemented but follow the same architectural pattern as domain agents.

10.10.4 Architectural Principle

The critical property of this framework is that no agent bypasses the well-posedness checker. Domain agents reduce the number of refinement iterations by teaching the LLM discipline-specific patterns, but they do not need to be correct in every detail: the checker catches what they miss. An arbitrarily creative (and arbitrarily unreliable) upstream agent is safe to deploy because the checker is the formal backstop, and only checked specifications enter the qualified code generation pipeline. This separation of creative generation from formal verification is the same principle that governs the single-agent elicitation loop, extended to a richer set of participants.

11 Summary

CBL addresses a specific gap in the MBPD pipeline: the unverified translation from natural-language requirements to executable models. The framework has been validated on the traffic signal examples (Sections 3 and 8) through round-trip trace comparison between compiled Stateflow models and Python reference implementations. The CBL compiler and three-layer NLP pipeline (LLM extraction, OCaml reasoning via `cblc reason`, compiler construction) demonstrate end-to-end specification from structured facts to well-posed CBL. Six agents and a coordinating skill implement the agentic specification framework, with an FDI domain agent as the first domain-specific knowledge carrier.

The framework enforces correctness through multiple independent mechanisms. At the specification level, well-posedness checking (guard exclusivity, completeness, action totality, invariant consistency) catches structural defects, while behavioral coverage criteria apply MC/DC discipline before any model exists. Between the LLM and the compiler, a defense-in-depth trust architecture (six trust zones, three validation gates) prevents untrusted output from reaching the compiler without schema validation, provenance enforcement, and verdict verification. After the specification passes the checker, six deterministic hallucination mitigation modules address semantic fidelity: back-translation, dual extraction, mutation testing, pattern justification, provenance control, and traceability. The Z3 guard checker, integrated into the compiler, strengthens these checks with SMT-based verification of guard exclusivity and completeness. Throughout the pipeline, a provenance system tracks the epistemic status of every fact and blocks unconfirmed LLM inferences from entering the final specification.

Compilation to Stateflow and SCADE preserves trace equivalence: the compiled model produces identical outputs and mode sequences as the CBL specification for all input sequences. This property is validated empirically via round-trip simulation; a formal proof via structural induction is planned. The full traceability chain from requirements through CBL clauses, IR elements, model artifacts, and generated code supports DO-178C and ISO 26262 certification arguments.

Large language models accelerate the specification process through interactive elicitation and refinement, but they do not generate code, perform verification, or make design decisions. The well-posedness checker catches specification-level errors, regardless of their origin, before they propagate into the qualified pipeline. The boundary is deliberate: LLMs contribute where they add value (natural-language understanding) and are excluded where they introduce unacceptable risk (safety-critical code generation).

What remains is an account of scale: industrial validation on requirement sets of realistic size, a controlled experimental evaluation, hardened SMT guard analysis, and a SCADE backend. The framework is ready for that test.

References

[1] Owura Asare, Meiyappan Nagappan, and N. Asokan. A user-centered security evaluation of Copilot. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering (ICSE)*, 2024.

[2] J. W. Backus, R. J. Beeber, S. Best, R. Goldberg, L. M. Haibt, H. L. Herrick, R. A. Nelson, D. Sayre, P. B. Sheridan, H. Stern, I. Ziller, R. A. Hughes, and R. Nutt. The FORTRAN automatic coding system. In *Proceedings of the Western Joint Computer Conference*, pages 188–198, February 1957.

[3] Albert Benveniste, Benoît Caillaud, Dejan Nickovic, Roberto Passerone, Jean-Baptiste Raclet, Philipp Reinkemeier, Alberto Sangiovanni-Vincentelli, Werner Damm, Thomas A. Henzinger, and Kim G. Larsen. Contracts for system design. *Foundations and Trends in Electronic Design Automation*, 12(2–3):124–400, 2018.

[4] Zhengkun Di, Chaoli Zhang, Hongtao Lv, Lizhen Cui, and Lei Liu. LoRP: LLM-based logical reasoning via Prolog. *Knowledge-Based Systems*, 327:114140, 2025.

[5] Dimitra Giannakopoulou, Anastasia Mavridou, Julian Rhein, Thomas Pressburger, Johann Schumann, and Nija Shi. Formal requirements elicitation with FRET. In *Proceedings of the International Working Conference on Requirements Engineering: Foundation for Software Quality (REFSQ)*, 2020. NASA Technical Reports Server, Document ID 20200001989.

[6] C. Heitmeyer. SCR: a practical method for requirements specification. In *17th DASC. AIAA/IEEE/SAE Digital Avionics Systems Conference*, pages C44/1–C44/5, 1998.

[7] John E. Hopcroft, Rajeev Motwani, and Jeffrey D. Ullman. *Introduction to Automata Theory, Languages, and Computation.* Addison-Wesley, 3rd edition, 2006.

[8] ISO. ISO 26262: Road vehicles — functional safety, 2018. International Standard, 2nd edition.

[9] JUXT Ltd. Allium: A behavioural specification language. `https://juxt.github.io/allium/`, 2026. Accessed March 2026.

[10] K. Keutzer, A. R. Newton, J. M. Rabaey, and A. Sangiovanni-Vincentelli. System-level design: orthogonalization of concerns and platform-based design. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 19(12):1523–1543, 2000.

[11] Robyn R. Lutz. Analyzing software requirements errors in safety-critical, embedded systems. In *Proceedings of the IEEE International Symposium on Requirements Engineering*, pages 126–133, 1993.

[12] Alistair Mavin, Philip Wilkinson, Adrian Harwood, and Mark Novak. Easy approach to requirements syntax (EARS). In *Proceedings of the 17th IEEE International Requirements Engineering Conference (RE)*, pages 317–322, 2009.

[13] Hammond Pearce, Baleegh Ahmad, Benjamin Tan, Brendan Dolan-Gavitt, and Ramesh Karri. Asleep at the keyboard? assessing the security of GitHub Copilot's code contributions. In *2022 IEEE Symposium on Security and Privacy (SP)*, pages 754–768, 2022.

[14] Neil Perry, Megha Srivastava, Deepak Kumar, and Dan Boneh. Do users write more insecure code with AI assistants? In *Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, pages 2785–2799, 2023.

[15] Ashwin Rao. The future of programming: From problem specifications to AI-compiled implementations — a type-theoretic foundation. Technical essay, Stanford ICME, March 2026. Accessed March 2026.

[16] L. Rierson. *Developing Safety-Critical Software: A Practical Guide for Aviation Software and DO-178C Compliance*. CRC Press, 2013.

[17] Alberto Sangiovanni-Vincentelli. Quo vadis, SLD? reasoning about the trends and challenges of system level design. *Proceedings of the IEEE*, 95(3):467–506, 2007.

[18] Michael Sipser. *Introduction to the Theory of Computation*. Cengage Learning, 3rd edition, 2013.

[19] Burak Yetiştiren, Işık Özsoy, Miray Ayerdem, and Eray Tüzün. Evaluating the code quality of AI-assisted code generation tools: An empirical study on GitHub Copilot, Amazon CodeWhisperer, and ChatGPT. *arXiv preprint arXiv:2304.10778*, 2023.

[20] Beiqi Zhang, Peng Liang, Xiyu Zhou, Aakash Ahmad, and Muhammad Waseem. Practices and challenges of using GitHub Copilot: An empirical study. In *Proceedings of the 35th International Conference on Software Engineering and Knowledge Engineering (SEKE)*, 2023.

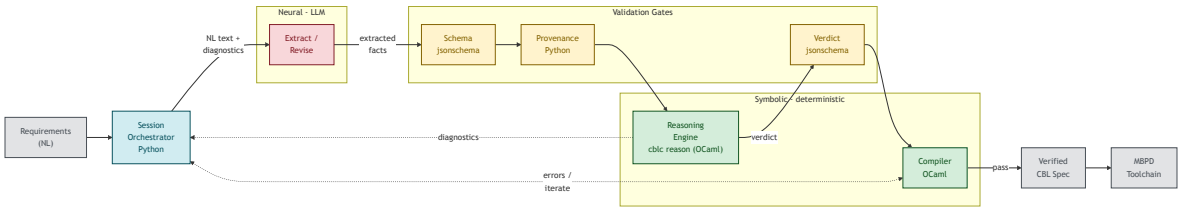

Figure 3: CBL pipeline architecture. Red: untrusted neural component (LLM). Yellow: validation gates (deterministic). Green: symbolic engines (deterministic). Gray: external systems. The session orchestrator manages the iteration loop, feeding diagnostics back to the LLM until the specification passes all gates.